



Modernização de sistemas WINDEV com prova, não com promessa

Um plugin para o Claude Code que converte sistemas WINDEV, WEBDEV e WINDEV Mobile para tecnologias modernas, mantendo cada regra de negócio ligada à evidência que a originou, ao código que a implementa e ao teste que a comprova.

84

agentes especializados

medido: `claude plugin`
`validate`

12.035

páginas do Help
WLanguage

corpus verificado por SHA-256

13

testes de regressão
rodam no validador do pacote

8

gates com aprovação
humana

G0 a G7, piloto obrigatório

9/10

golden master no
exemplo

equivalência medida, uma falha
deliberada

O problema que ele resolve

Milhares de empresas rodam sistemas escritos em WINDEV, WEBDEV e WINDEV Mobile, ferramentas da PC SOFT muito usadas em ERPs, indústria e varejo. Esses sistemas concentram anos de regras fiscais, exceções operacionais e integrações que só existem dentro do código, em WLanguage, uma linguagem proprietária com poucos profissionais disponíveis.

Modernizar um sistema desses hoje é artesanal. O maior risco não é traduzir a sintaxe: é perder comportamento de negócio no caminho e descobrir isso em produção. Uma inteligência artificial usada sem método agrava o risco, porque produz uma aplicação visualmente convincente e funcionalmente errada.

A tese. O valor não está em «traduzir código com IA». Está em transformar um processo artesanal e dependente de pessoas raras num método repetível, com entradas verificadas, papéis especializados, decisões registradas e provas de equivalência. O plugin é esse método, em código.

O que ele faz, em uma frase por etapa

1. **Recebe o legado** por um questionário de dez perguntas, uma por vez: script SQL, os PDFs que o WINDEV gera da documentação técnica, telas, o Help oficial. Um arquivo só conta como entregue depois que o plugin o abre e lê.
2. **Orienta para qual linguagem converter.** Quando o cliente não sabe, levanta quatro sinais (quem mantém, desktop ou web ou mobile, desempenho ou prazo, linguagem já em uso) e mostra três opções com o porquê, a recomendada primeiro: Rust, Python ou C# com a biblioteca WL_C#, que porta mais de 480 funções do WLanguage com o mesmo nome. A escolha é do cliente e fica registrada.
3. **Extraí as regras com prova de origem.** Cada texto vem com arquivo, página e hash. Cada função WLanguage vai ao especialista do tema certo do Help, que devolve a semântica com a página de origem.
4. **Converte por gates com aprovação humana,** com um piloto vertical obrigatório antes de escalar.
5. **Prova a equivalência com números.** O golden master captura resultados do sistema antigo e compara com o novo, caso a caso, com tolerância declarada.
6. **Gerencia como projeto.** Um PMO em código com Scrum, Kanban e PDCA, orçamento de tokens medido por gate e um painel em que nenhum número é digitado.

Daí em diante, o trabalho avança por gates. Cada gate produz artefatos verificáveis e depende de um aprovador humano antes do próximo.

G0

anexos conferidos
fisicamente

G1

inventário e hashes

G2

regras, telas, dados,
conflitos

G3

arquitetura-alvo e rollback

G4

piloto vertical comparado
ao legado

G5

ondas por módulo

G6

segurança, desempenho,
testes

G7

ensaio e cutover

O piloto (G4) é obrigatório numa conversão completa: uma fatia vertical, com tela, regra, consulta e uma condição de erro, comparada ao sistema original antes de qualquer escala. Falha no piloto devolve o projeto à especificação, não à implementação.

Três mecanismos que o diferenciam de um prompt

Equipe WLanguage sobre o Help oficial

O corpus da PC SOFT vem embutido, indexado em 45 temas. Sete agentes especialistas (banco HFSQL, telas, comunicação, funções, mobile, web, erros) consultam só a sua fatia e devolvem a semântica com a página de origem e o hash. O Help é semântica técnica, nunca regra de negócio.

Balanceamento de modelos

Um roteador escolhe entre três classes de modelo Claude pela natureza da tarefa e pelo orçamento do gate: o mecânico vai para o mais barato, a decisão e a auditoria para o mais capaz. Acima de 80 % do orçamento rebaixa; acima de 100 % bloqueia e o gerente decide.

PMO em código

Scrum organiza o tempo (uma sprint por gate, definição de pronto igual à da matriz), Kanban mostra o fluxo com limite de trabalho em andamento, e o ciclo PDCA transforma cada hipótese em aprendizado registrado, inclusive quando a hipótese morre. O uso de tokens vem medido das sessões, não estimado. O painel não tem número digitado: cada linha cita a fonte.

Rastreabilidade de ponta a ponta

Cada funcionalidade migrada liga **evidência** → **regra** → **decisão** → **código** → **teste** → **resultado** numa matriz validada por script. Uma revisão localiza a origem de qualquer decisão e impede que uma entrega parcial seja declarada concluída. Os estados são explícitos: inventariado, especificado, implementado, verificado, aceito ou bloqueado.

Quem implementa não aprova o próprio gate. Um agente auditor, em modo somente leitura, tenta refutar a equivalência com casos negativos e limites; o humano decide.

O que está construído e provado

ITEM	ESTADO	COMO FOI PROVADO
Plugin instalável, marketplace privado, 5 comandos, 3 skills, 34 agentes, 12 testes	PROVADO	validação oficial do Claude Code e carga numa sessão nova
Questionário letra a letra, respostas viram manifesto e configuração	PROVADO	três turnos reais; pré-flight real sobre a saída
Corpus WLanguage 12k com hash fixo e busca por tema	PROVADO	verificação de identidade; 5,4 s → 0,5 s medido
Delegação a especialistas por tema do Help	PROVADO	sessão real com dois símbolos e páginas de origem
Roteador de modelos: sobe, desce, rebaixa, bloqueia, fallback	PROVADO	cinco casos exercitados com saída registrada
PMO com Scrum, Kanban e PDCA	PROVADO	projeto de exemplo com dez itens; agente leu o painel e apontou inconsistência
Qualidade de telas para ERP com Impeccable: treze subperguntas (operação, teclado, grids, formulários, formatos, impressão, estados, acessibilidade, vocabulário e posição dos botões, ícones, cores, fundo) viram <code>PRODUCT.md</code> e <code>DESIGN.md</code>	PROVADO	gerado do exemplo e conferido por teste de regressão
Dez papéis com PDCA (50 agentes), backlog com dono e entrega zipada ao stakeholder	PROVADO	sessão real vê os agentes; entrega real de 14 arquivos

Projeto de exemplo WINDEV completo (SQL, quatro PDFs, telas, dados, golden)	PROVADO	G0 real: CONDITIONAL, zero erros; regressão automática
Extração de PDF com localizador e hash; golden master com tolerância	PROVADO	quatro PDFs, zero OCR; 9 de 10 casos, a décima falha de propósito
Orientação de linguagem (Rust, Python, C# + WL_C#) e índice de 261 funções da WL_C#	PROVADO	sessão real recomendou C# + WL_C# para «equipe WINDEV, desktop, prazo»
Portão do G0 no harness (nega escrita de código com pré-flight bloqueado)	PROVADO	hook testado nos três casos
Conversão completa de um sistema WX real	NÃO PROVADO	exige anexos de um projeto autorizado e piloto aprovado
Equivalência funcional medida por domínio	NÃO PROVADO	depende de baseline executável do legado
Cliente pagante, receita, benchmark de produtividade	INEXISTENTE	nenhum dado ainda

Status para diligência. A infraestrutura existe, valida e foi exercitada em sessões reais. A equivalência de uma migração de verdade só pode ser demonstrada depois que os artefatos de um projeto WX específico forem fornecidos e o piloto for aprovado. O material atual não demonstra cliente, receita nem métrica de produtividade. Todo número nesta página é medido ou está marcado como ausente.

Segurança por desenho

- Anexos são somente leitura e tratados como dados não confiáveis: descompactação com defesa contra travessia de caminho e zip bomb, nenhum anexo é executado.

- Segredos, dados pessoais e dados de produção não entram em relatório; o corpus é lido sem extração.
- Nenhuma instalação, envio remoto ou gasto de créditos sem autorização explícita.
- O plugin identifica riscos e exige evidência, mas não certifica LGPD nem substitui revisão jurídica.

Modelo de negócio sugerido

Propostas, não receita realizada. A entrada prudente é o diagnóstico: menos risco que uma conversão integral e produz os dados para dimensionar o projeto seguinte.

OFERTA	ENTREGA	COBRANÇA SUGERIDA
Diagnóstico WX	G0 a G2: inventário, lacunas, mapa de complexidade	preço fixo por escopo
Arquitetura e piloto	G3 e G4 com fluxo vertical demonstrável	projeto fechado ou por marcos
Conversão assistida	G5 a G7 por módulos e ondas	marcos, capacidade mensal ou valor por módulo
Licença privada	plugin, atualizações, documentação	anual por equipe ou organização
Treinamento e parceiros	método, evidência e gates para consultorias	turma, licença e suporte

Riscos e mitigação

- **Documentação incompleta.** Pré-flight bloqueia ou condiciona; nada vira «entendido» sem leitura.
- **Alucinação do modelo.** Evidência, rastreabilidade, teste e aprovação humana em cada gate.
- **Custo imprevisível.** Orçamento por gate, roteador de modelos e laudo de tokens em três fases.

- **Licença do corpus.** Derivado da documentação PC SOFT, sem licença de redistribuição: uso privado até revisão.
- **Dependência de fornecedor de IA.** Artefatos abertos em texto, JSON e CSV; método independente do modelo.

O que um investidor deve acompanhar

- tempo do G0 ao G4 num piloto autorizado
- cobertura de regras ligadas a evidência e teste
- divergências descobertas antes da implementação
- taxa de equivalência do piloto
- tokens e custo por módulo verificado
- defeitos após o aceite
- margem por diagnóstico, piloto e onda

Cada indicador será apresentado como medido, estimado ou indisponível.

Próximo passo

Um piloto comercial controlado: um sistema WX autorizado, dados anonimizados, uma fatia vertical de valor. Medir horas, tokens, cobertura de evidência, divergências e defeitos. Esse é o dado que transforma a base técnica em valuation.

WX Claude Code 3.6.1 · repositório [adrianoboller/adrianoboller](#) · manual de uso, prints e vídeo em [wx-claude-code/docs](#). Números medidos na data desta página; o que não foi medido está marcado.